

TEAMHUB · V1

Schéma de base de données

Postgres (Laravel) · MongoDB (Realtime) · Flux inter-modules

Projet	TeamHub (Excellence Team)
Version	V1 — 2026-10-01
Branche	claude/affectionate-fermi-hskazy
Généré depuis	Migrations + Modèles Eloquent + Collections Mongo



M0 — Socle (users, orgs, membres)



M1 — Projets & Tâches



M3 — CRM / BizDev



M4 — Social (fil interne)

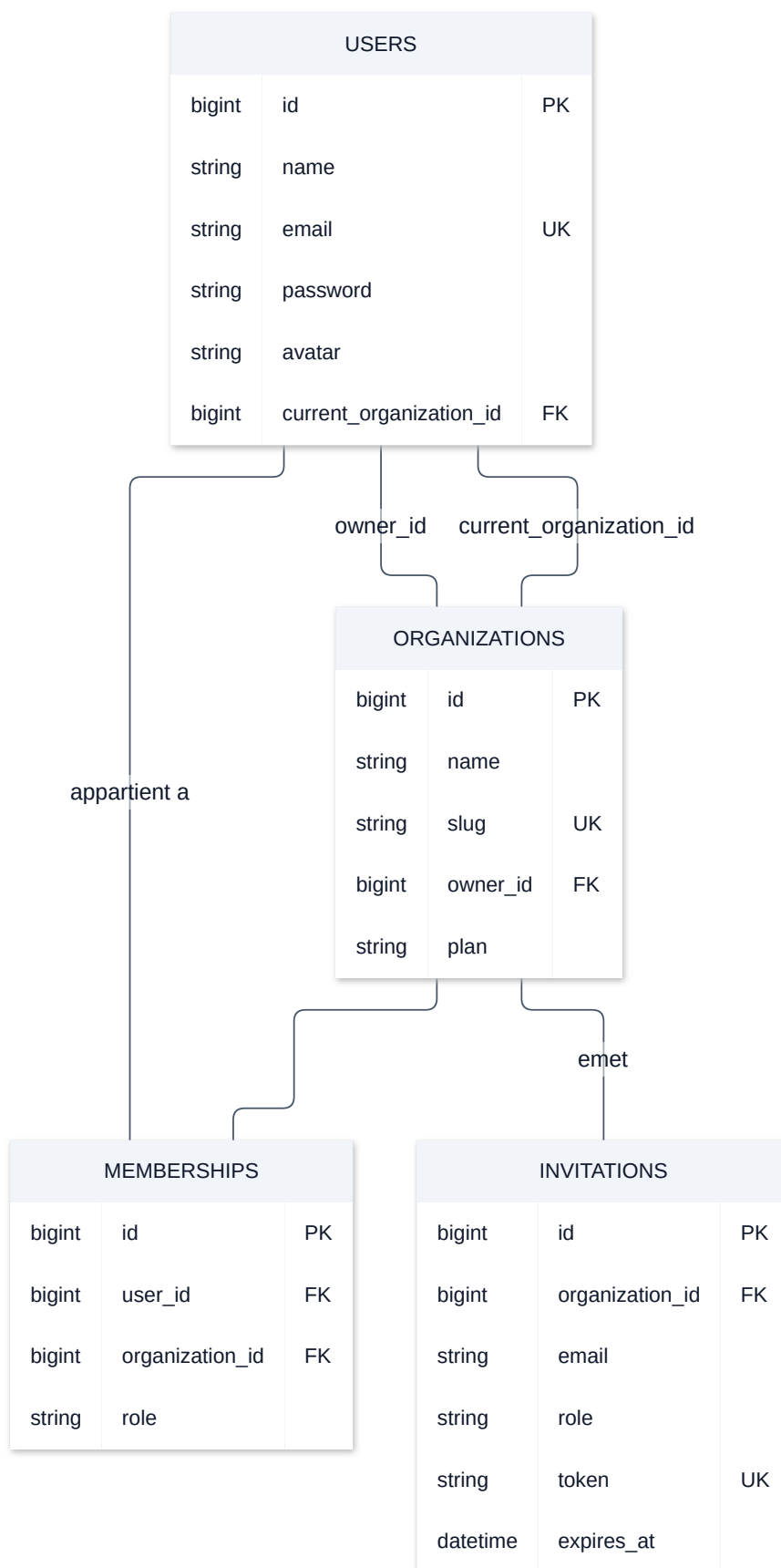


Transverses
(attachments, activities)



MongoDB — Chat realtime

Multi-tenant : un utilisateur peut appartenir à plusieurs organisations ; chaque appartenance porte son rôle. Toutes les tables métier portent ensuite `organization_id` injecté automatiquement par le trait `BelongsToOrganization` .



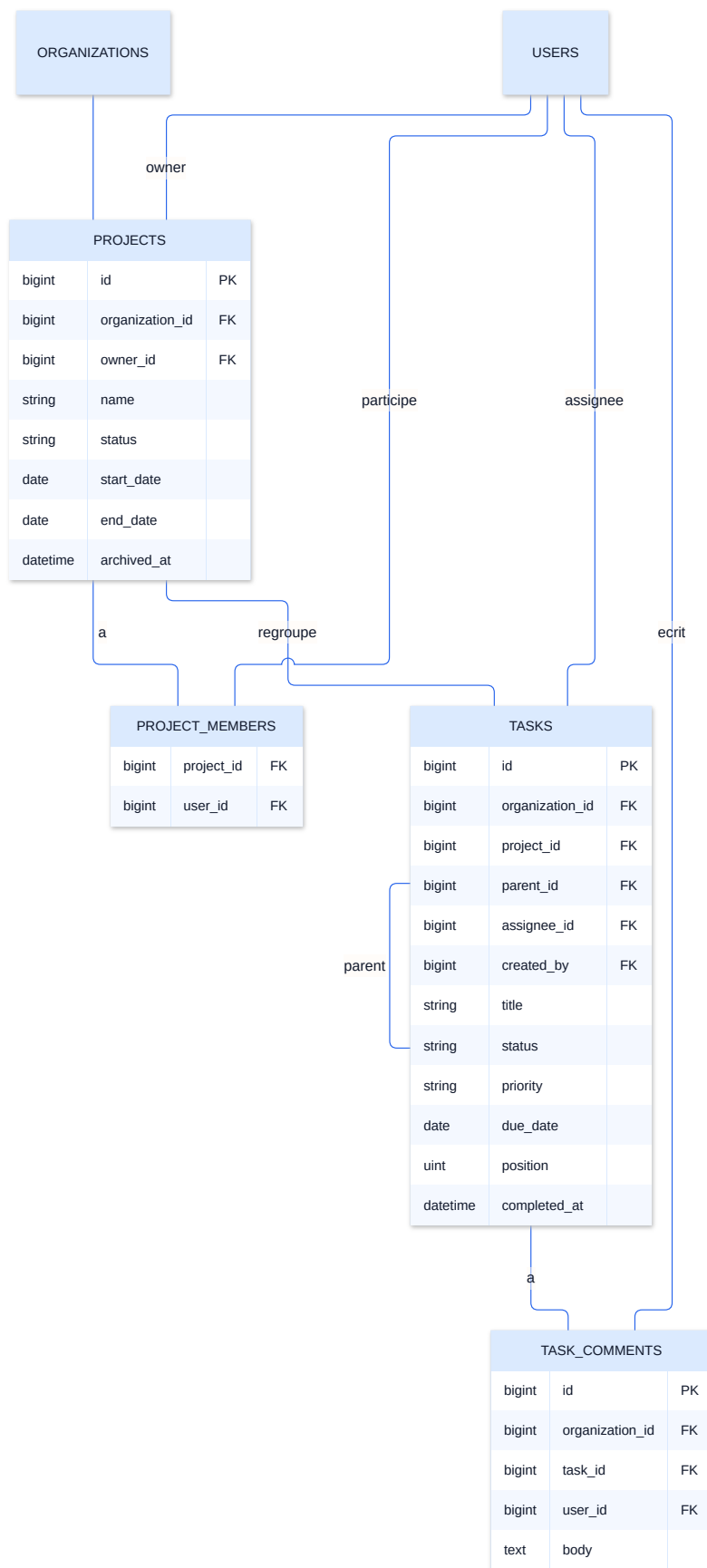
Rôles possibles (colonne `memberships.role`)

RÔLE	CAPACITÉS
------	-----------

owner	Tout, y compris suppression de l'organisation
admin	Membres, projets, paramètres (pas supprimer l'org)
manager	Crée et pilote ses projets, voit son CRM et ses analytics
member	Avance ses tâches, chat, social — ni CRM ni analytics
guest	Lecture d'un projet partagé seulement

Projets, membres, tâches, commentaires

`parent_id` permet les sous-tâches. `assignee_id` est validé serveur-side comme membre de l'organisation courante (empêche l'affectation cross-org).

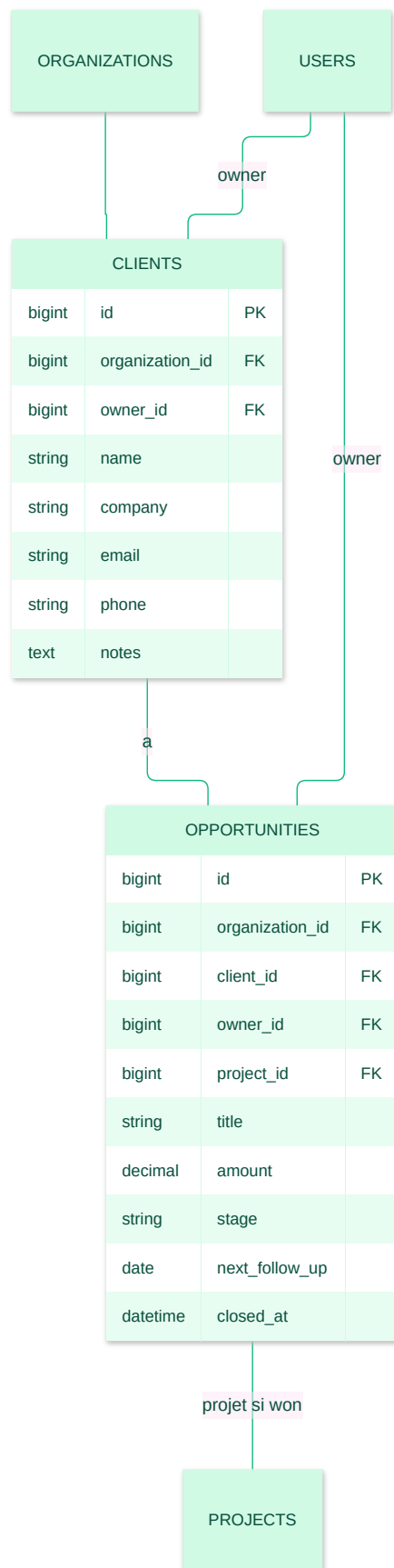


Statuts

TABLE	COLONNE	VALEURS
-------	---------	---------

projects	status	upcoming , in_progress , on_hold , done
tasks	status	todo , in_progress , review , done
tasks	priority	low , normal , high , urgent

Règle métier clé : quand `opportunities.stage` passe à `won`, un `project` est **créé automatiquement** et lié à l'opportunité via `project_id`.

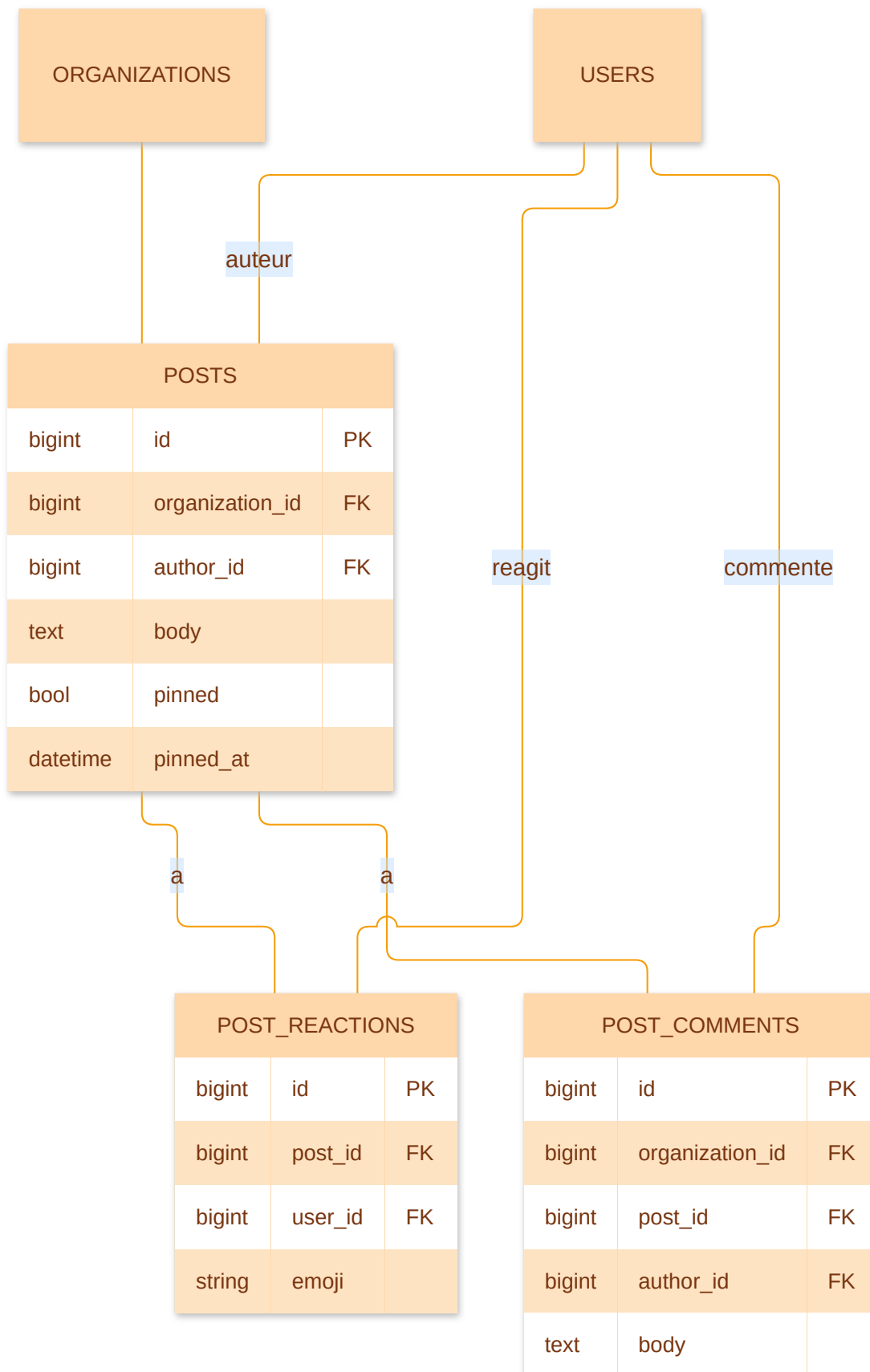


Pipeline (colonne `opportunities.stage`)

ÉTAPE	SIGNIFICATION
-------	---------------

prospect	Identifié, pas encore contacté
contacted	Premier contact établi
proposal	Devis / proposition envoyée
won	Signé — crée un projet automatiquement
lost	Perdu

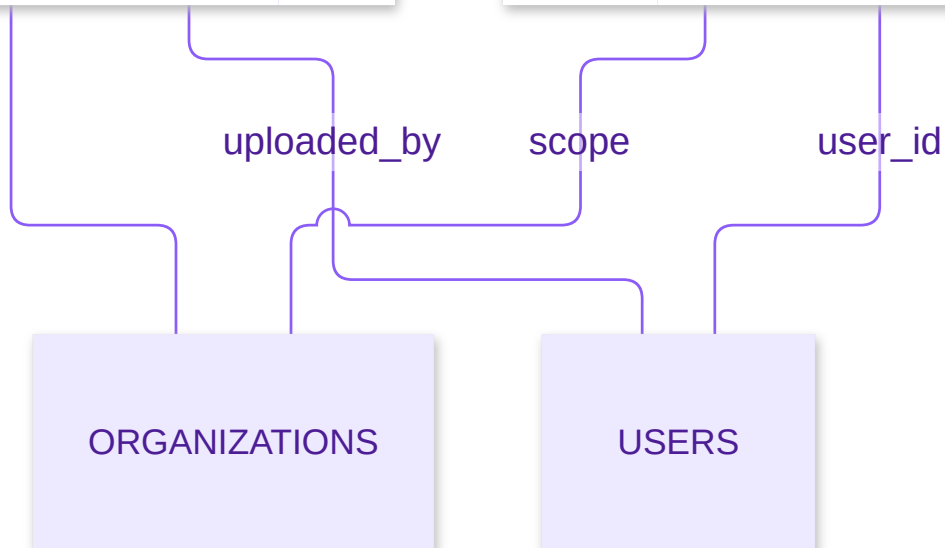
Posts épinglés réservés aux `owner` / `admin` . Réaction unique par triplet (post, user, emoji).



Deux tables polymorphes qui s'attachent à n'importe quelle entité métier via `*_type` + `*_id`.

ATTACHMENTS		
bigint	id	PK
bigint	organization_id	FK
bigint	uploaded_by	FK
string	attachable_type	
bigint	attachable_id	
string	path	
string	name	
string	mime	
bigint	size	

ACTIVITIES		
bigint	id	PK
bigint	organization_id	FK
bigint	user_id	FK
string	subject_type	
bigint	subject_id	
string	action	
json	meta	



Exemples d'actions tracées (`activities.action`)

ACTION	DÉCLENCHÉE PAR
client.created	POST /clients
opportunity.stage_changed	PATCH /opportunities/{id} (changement de stage)
project.created_from_opportunity	Auto quand une opportunité passe à won

Écrites par `apps/realtime` (Node.js). Les canaux projets sont synchronisés depuis Laravel via Redis pub/sub (`wine:project:events`). `organization_id` répliqué dans chaque document pour l'isolation.

CHANNELS		
objectid	id	PK
int	organization_id	
string	type	
int	project_id	
array	member_ids	
string	name	

MESSAGES		
objectid	id	PK
int	organization_id	
objectid	channel_id	FK
int	sender_id	
text	body	
array	attachments	
array	read_by	
datetime	created_at	

Index

`messages` : index composé (`organization_id, channel_id, created_at desc`) — chargement rapide du fil le plus récent d'un canal.

Les tables ne vivent pas en silos. Trois chemins de communication structurent le système : création d'entités, notifications temps réel, agrégation analytics.

